Homework 3 :: MATH 5152

Due: Monday, February 9 at 11:59 AM

Submission instructions: Submit a single PDF file named `LASTNAME-hw3.pdf`. All solutions must be typed in LaTeX and uploaded to Canvas before the deadline.

1. Consider the linear system

$$A = \begin{bmatrix} 3 & -1 & 2 \\ -1 & 5 & 1 \\ 2 & 1 & 4 \end{bmatrix}, \qquad b = \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix}.$$

(a) Determine whether A is *diagonally dominant* by rows, by columns, or both.

$$|3| \geq |-1| + |2| = 3 \checkmark$$
$$|5| \geq |-1| + |1| = 2 \checkmark$$
$$|4| \geq |2| + |1| = 3 \checkmark$$

A is therefore diagonally dominant by rows.

$$|3| \geq |-1| + |2| = 3 \checkmark$$
$$|5| \geq |-1| + |1| = 2 \checkmark$$
$$|4| \geq |2| + |1| = 3 \checkmark$$

A is also diagonally dominant by columns.

(b) Starting from $x^{(0)} = [1,\, 1,\, 0]^{\mathsf{T}}$, perform **two** iterations of the *Jacobi method* to approximate the solution. Report the error $\|x^{(k)} - x^*\|_\infty$ for $k = 1, 2$, where x^* is the exact solution.

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right)$$

First, we compute the exact solution $x^* = A^{-1}b$:

$$x^* = A^{-1}b = \begin{bmatrix} 3 & -1 & 2 \\ -1 & 5 & 1 \\ 2 & 1 & 4 \end{bmatrix}^{-1} \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} -\frac{30}{29} \\ -\frac{11}{29} \\ \frac{25}{29} \end{bmatrix} \approx \begin{bmatrix} -1.034 \\ -0.379 \\ 0.862 \end{bmatrix}$$

Given initial guess $x^{(0)} = [1, 1, 0]^T$, we perform two iterations.

Iteration 1:

$$x_1^{(1)} = \frac{1}{3}\left(-1 - (-1)(1) - 2(0)\right) = \frac{1}{3}(-1+1) = 0$$
$$x_2^{(1)} = \frac{1}{5}\left(0 - (-1)(1) - 1(0)\right) = \frac{1}{5}(1) = 0.2$$
$$x_3^{(1)} = \frac{1}{4}\left(1 - 2(1) - 1(1)\right) = \frac{1}{4}(-2) = -0.5$$

Therefore, $x^{(1)} = [0, 0.2, -0.5]^T$.
The error is:

$$\|x^{(1)} - x^*\|_\infty = \max\left\{\left|0 - \left(-\frac{30}{29}\right)\right|, \left|0.2 - \left(-\frac{11}{29}\right)\right|, \left|-0.5 - \frac{25}{29}\right|\right\} = \frac{79}{58}$$

Iteration 2:

$$x_1^{(2)} = \frac{1}{3}\left(-1 - (-1)(0.2) - 2(-0.5)\right) = \frac{1}{3}(-1 + 0.2 + 1) = \frac{1}{15}$$
$$x_2^{(2)} = \frac{1}{5}\left(0 - (-1)(0) - 1(-0.5)\right) = \frac{1}{5}(0.5) = 0.1$$
$$x_3^{(2)} = \frac{1}{4}\left(1 - 2(0) - 1(0.2)\right) = \frac{1}{4}(0.8) = 0.2$$

Therefore, $x^{(2)} = \left[\frac{1}{15}, 0.1, 0.2\right]^T$.
The error is:

$$\|x^{(2)} - x^*\|_\infty = \max\left\{\left|\frac{1}{15} - \left(-\frac{30}{29}\right)\right|, \left|0.1 - \left(-\frac{11}{29}\right)\right|, \left|0.2 - \frac{25}{29}\right|\right\} = \frac{479}{435}$$

(c) Using the same initial guess, perform **two** iterations of the *Gauss–Seidel method*. Again report the error $\|x^{(k)} - x^*\|_\infty$ for $k = 1, 2$.

Iteration 1:

$$x_1^{(1)} = \frac{-1 + 1 - (2)(0)}{3} = 0$$
$$x_2^{(1)} = \frac{0 - 0}{5} = 0$$
$$x_3^{(1)} = \frac{1 - (2)(0) - 0}{4} = \frac{1}{4}$$

Therefore, $x^{(1)} = \left[0, 0, \frac{1}{4}\right]^T$.
The error is:

$$\|x^{(1)} - x^*\|_\infty = \max\left\{\left|0 - \left(-\frac{30}{29}\right)\right|, \left|0 - \left(-\frac{11}{29}\right)\right|, \left|\frac{1}{4} - \frac{25}{29}\right|\right\} = \frac{30}{29}$$

Iteration 2:

$$x_1^{(2)} = \frac{-1 + 0 - (2)(1/4)}{3} = \frac{-3/2}{3} = -\frac{1}{2}$$
$$x_2^{(2)} = \frac{-1/2 - 1/4}{5} = \frac{-3/4}{5} = -\frac{3}{20}$$
$$x_3^{(2)} = \frac{1 - 2(-1/2) - (-3/20)}{4} = \frac{43}{80}$$

Therefore, $x^{(2)} = \left[-\frac{1}{2}, -\frac{3}{20}, \frac{43}{80}\right]^T$.

The error is:

$$\|x^{(2)} - x^*\|_\infty = \max\left\{\left|-\frac{1}{2} - \left(-\frac{30}{29}\right)\right|, \left|-\frac{3}{20} - \left(-\frac{11}{29}\right)\right|, \left|\frac{43}{80} - \frac{25}{29}\right|\right\} = \frac{31}{58}$$

2. **(Coding Problem)** Using the same system and initial vector $x^{(0)} = [1, 1, 0]^{\mathrm{T}}$, implement the following iterative methods in Python with tolerance $\varepsilon = 10^{-4}$:

   (a) Jacobi method

   (b) Gauss–Seidel method

   (c) Successive Over–Relaxation (SOR) method

   For SOR, run your code with two relaxation parameters $\omega = 0.7$ and $\omega = 1.5$.

   For each method (and each value of ω in SOR) report:

   (a) the number of iterations required to satisfy the stopping criterion

   $$\|x^{(k)} - x^*\|_\infty \leq \varepsilon,$$

   (b) the final approximation $x^{(k)}$,

   (c) the final error $\|x^{(k)} - x^*\|_\infty$, where $x^* = A^{-1}b$ is the exact solution.

   Jacobi Method:

```python
import numpy as np

A = np.array([[3, -1, 2],
              [-1, 5, 1],
              [2, 1, 4]], dtype=float)
b = np.array([-1, 0, 1], dtype=float)
x0 = np.array([1, 1, 0], dtype=float)
epsilon = 1e-4

x_star = np.linalg.solve(A, b)
print("x*:", x_star)

def jacobi(A, b, x0, epsilon):
    n = len(b)
    x = x0.copy()

    for k in range(max_iter):
        x_new = np.zeros(n)

        for i in range(n):
            sum_val = sum(A[i][j] * x[j] for j in range(n) if j
                != i)
            x_new[i] = (b[i] - sum_val) / A[i][i]

        error = np.linalg.norm(x_new - x_star, np.inf)
```

```
        if error <= epsilon:
            return x_new, k + 1, error

        x = x_new.copy()

    return x, max_iter, np.linalg.norm(x - x_star, np.inf)

x_jacobi, iter_jacobi, error_jacobi = jacobi(A, b, x0, epsilon)

print("(a) number of iterations required:", iter_jacobi)
print("(b) final approximation x^(k):", x_jacobi)
print("(c) final error ||x^(k) - x*||_inf:", error_jacobi)
```

```
x*: [-1.03448276 -0.37931034  0.86206897]
(a) number of iterations required: 32
(b) final approximation x^(k): [-1.03438966 -0.37926347  0.86198974]
(c) final error ||x^(k) - x*||_inf: 9.310116875571595e-05
```

Gauss-Seidel Method:

```
def gauss_seidel(A, b, x0, x_star, epsilon, max_iter):
    n = A.shape[0]
    x = x0.copy()
    k = 0

    while k < max_iter:
        x_new = x.copy()

        for i in range(n):
            s1 = A[i, :i] @ x_new[:i]
            s2 = A[i, i+1:] @ x[i+1:]
            x_new[i] = (b[i] - s1 - s2) / A[i, i]

        error = np.linalg.norm(x_new - x_star, np.inf)

        if error <= epsilon:
            return k + 1, x_new, error

        x = x_new
        k += 1

    return k, x, error

kG, xG, eG = gauss_seidel(A, b, x0, x_star, epsilon, max_iter)
print("gauss-seidel:")
print("x*:", x_star)
print("number of iterations required:", kG)
print("final approximation x^(k):", xG)
print("final error ||x^(k) - x*||_inf:", eG)
```

```
gauss-seidel:
x*: [-1.03448276 -0.37931034  0.86206897]
number of iterations required: 17
final approximation x^(k): [-1.03442376 -0.37928542  0.86203324]
final error ||x^(k) - x*||_inf: 5.899542540710456e-05
```

Successive Over-Relaxation (SOR) Method:

```python
def sor(A, b, x0, x_star, omega, epsilon, max_iter):
    n = A.shape[0]
    x = x0.copy()
    k = 0

    while k < max_iter:
        x_new = x.copy()

        for i in range(n):
            s1 = A[i, :i] @ x_new[:i]
            s2 = A[i, i+1:] @ x[i+1:]

            x_gs = (b[i] - s1 - s2) / A[i, i]
            x_new[i] = (1 - omega) * x[i] + omega * x_gs

        error = np.linalg.norm(x_new - x_star, np.inf)

        if error <= epsilon:
            return k + 1, x_new, error

        x = x_new
        k += 1

    return k, x, error

kS1, xS1, eS1 = sor(A, b, x0, x_star, 0.7, epsilon, max_iter)
print("SOR (omega=0.7):")
print("x*:", x_star)
print("number of iterations required:", kS1)
print("final approximation x^(k):", xS1)
print("final error ||x^(k) - x*||_inf:", eS1)
print()

kS2, xS2, eS2 = sor(A, b, x0, x_star, 1.5, epsilon, max_iter)
print("SOR (omega=1.5):")
print("x*:", x_star)
print("number of iterations required:", kS2)
print("final approximation x^(k):", xS2)
print("final error ||x^(k) - x*||_inf:", eS2)
```

```
SOR (omega=0.7):
x*: [-1.03448276 -0.37931034  0.86206897]
```

```
number of iterations required: 34
final approximation x^(k): [-1.03440665 -0.3792752   0.86201405]
final error ||x^(k) - x*||_inf: 7.610749420505769e-05

SOR (omega=1.5):
x*: [-1.03448276 -0.37931034  0.86206897]
number of iterations required: 15
final approximation x^(k): [-1.03445635 -0.37937805  0.86205696]
final error ||x^(k) - x*||_inf: 6.770883853046694e-05
```

As shown, the Gauss-Seidel method converges more quickly than the Jacobi method, with just 17 iterations compared to 32. On the other hand, SOR results in convergence within 34 iterations at $\omega = 0.7$ and 15 at $\omega = 1.5$. This is in line with the observation that SOR converges fastest when ω is well-chosen.